Views and Subqueries

Module Outcome:

- Name a query as a view so the next person does not rewrite a 20-line SELECT
- Nest a subquery when a join would be noisier
- Know a view is not a second copy of the data

Note: This chapter uses **MySQL** syntax. String literals use single quotes (`'value'`).

Views

A view can simply be considered a name to a SQL Query.

Create View

To create a view in the database, use the CREATE VIEW Statement.

Example: Create `user_base_details` view with id, name, age, gender and pincode.

```
CREATE VIEW user_base_details AS
SELECT id, name, age, gender, pincode
FROM user;
```

Note:

- In general, views are read only.
- We cannot perform write operations like updating, deleting and inserting rows in the base table through views.

Querying Using View

We can use its name instead of writing the original query to get the data.

```
SELECT *
FROM user_base_details;
```

List All Available Views

In MySQL, to list all the available views in the current database:

```
SELECT
    TABLE_NAME AS name
FROM
    information_schema.views
WHERE
    table_schema = DATABASE();
```

Alternative (shorter):

```
SHOW FULL TABLES WHERE Table_type = 'VIEW';
```

Output:

name
order_with_products
user_base_details

DELETE View

To remove a view from a database, use the DROP VIEW statement.

Syntax: `DROP VIEW view_name;`

Example:

```
DROP VIEW user_base_details;
```

Advantages of Views

1. Views are used to write complex queries that involve multiple joins, group by, etc., and can be used whenever needed.
2. Restrict access to the data such that a user can only see limited data instead of a complete table.

Subqueries

A **subquery** (also called an **inner query** or **nested query**) is a SQL query written inside another SQL query. The outer query uses the result of the inner query to filter, compute, or retrieve data.

Subqueries are commonly used when we need to compare values against aggregated results or against data from another related query.

Syntax (general form):

```
SELECT
    column1,
    column2
FROM
    table_name
WHERE
    column1 operator (SELECT column FROM table2 WHERE condition);
```

Note: The inner query must return a single value when used with comparison operators (`=`, `>`, `<`, etc.). When the inner query returns multiple values, use the `IN` operator.

Subquery in WHERE Clause

We use a subquery in the `WHERE` clause when the filter condition depends on the result of another query.

Example: Get all the products whose price is greater than the average price of all products.

```
SELECT
    name,
    price,
    brand
FROM
    product
WHERE
    price > (SELECT AVG(price) FROM product);
```

Output:

name	price	brand
Smart Watch	17000	Apple
Smart TV	40000	Sony
Blue Shirt	1000	puma
...	...	...

Example 2: Get all the products from brands that have at least one product with rating greater than 4.5.

```
SELECT
    name,
    price,
    brand,
    rating
FROM
    product
WHERE
    brand IN (
        SELECT brand
        FROM product
        WHERE rating > 4.5
    );
```

Output:

name	price	brand	rating
Smart Watch	17000	Apple	4.9
Bourbon Special	15	Britannia	4.6
Black shirt	600	puma	4.8
...	...	...	...

Subquery with Comparison Operators

A subquery can be used with comparison operators such as `=`, `>`, `<`, `>=`, and `<=` when the inner query returns a **single value**.

Syntax:

```
SELECT
    *
FROM
    table_name
WHERE
    column1 operator (SELECT aggregate_function(column2) FROM table_name);
```

Example: Get the details of the product that has the highest rating.

```
SELECT
    name,
    price,
    brand,
    rating
FROM
    product
WHERE
    rating = (SELECT MAX(rating) FROM product);
```

Output:

name	price	brand	rating
Smart Watch	17000	Apple	4.9

Note: If the inner query returns more than one row while using `=`, the query will result in an error. In such cases, use `IN` instead of `=`.

Subquery in SELECT Clause

We can use a subquery in the `SELECT` clause to compute a value for each row in the result.

Example: Get the name, price, and average price of products in the same category for every product.

```
SELECT
    name,
    category,
    price,
    (SELECT AVG(price) FROM product p2 WHERE p2.category = p1.category) AS avg_category_price
FROM
    product p1;
```

Output:

name	category	price	avg_category_price
Chocolate cake	Food	25	48.33
Blue Shirt	clothing	750	766.67

name	category	price	avg_category_price
Smart Watch	Gadgets	17000	16500
...	...	...	...

Note: When the inner query refers to columns from the outer query, it is called a **correlated subquery**. The inner query is executed once for each row of the outer query.

Correlated Subquery

In a **correlated subquery**, the inner query depends on the outer query. The inner query cannot be executed independently.

Example: Get the name and score of players who scored more than the average score of that player across all matches.

```
SELECT
    name,
    score,
    year
FROM
    player_match_details p1
WHERE
    score > (
        SELECT AVG(score)
        FROM player_match_details p2
        WHERE p2.name = p1.name
    );
```

Output:

name	score	year
Ram	75	2011
Lokesh	68	2012
Joseph	62	2010
...	...	...

Subquery in FROM Clause

A subquery in the `FROM` clause creates a temporary result set (derived table) that can be queried like a regular table. We use an **alias** to refer to this temporary table.

Syntax:

```
SELECT
    column1,
    column2
```

```
FROM
    (SELECT column1, column2 FROM table_name WHERE condition) AS alias_name;
```

Example: Get the brand and average rating of brands whose average rating is greater than 4.

```
SELECT
    brand,
    avg_rating
FROM
    (
        SELECT
            brand,
            AVG(rating) AS avg_rating
        FROM
            product
        GROUP BY
            brand
    ) AS brand_ratings
WHERE
    avg_rating > 4;
```

Output:

brand	avg_rating
Apple	4.9
puma	4.37
Britannia	4.23
...	...

Note: Always provide an alias when using a subquery in the `FROM` clause.

EXISTS and NOT EXISTS

`EXISTS` checks whether a subquery returns **any rows**. It returns `TRUE` if the subquery produces at least one row, otherwise `FALSE`. `NOT EXISTS` is the opposite.

Unlike `IN`, `EXISTS` stops as soon as it finds a matching row and is often more efficient for large datasets.

Syntax:

```
SELECT
    column1
FROM
    table_name
WHERE
    EXISTS (SELECT 1 FROM other_table WHERE condition);
```

Example 1: Get all products that have never been ordered (using a hypothetical `order_item` table).

```
SELECT
    name,
```

```
    price
FROM
    product p
WHERE
    NOT EXISTS (
        SELECT 1
        FROM order_item oi
        WHERE oi.product_id = p.id
    );
```

Output:

name	price
Smart TV	40000
Strawberry cake	60
...	...

Example 2: Get all brands that have at least one product with rating above 4.5.

```
SELECT DISTINCT
    brand
FROM
    product p1
WHERE
    EXISTS (
        SELECT 1
        FROM product p2
        WHERE p2.brand = p1.brand
        AND p2.rating > 4.5
    );
```

Output:

brand
Apple
Britannia
puma
...

Note: `EXISTS` / `NOT EXISTS` are standard SQL and work identically in MySQL.

Subquery vs JOIN

Both subqueries and joins can be used to combine data from multiple tables or queries.

Approach	When to use
Subquery	When filtering based on aggregated values or when the inner result is used as a condition
JOIN	When combining columns from multiple tables into a single result set

Example (same result using JOIN): Get products with price greater than average price.

```
SELECT
    p.name,
    p.price,
    p.brand
FROM
    product p
INNER JOIN (
    SELECT AVG(price) AS avg_price
    FROM product
) AS avg_table
ON p.price > avg_table.avg_price;
```

Note: Subqueries are easier to read for simple filters. Joins are often more efficient for combining data from multiple tables.